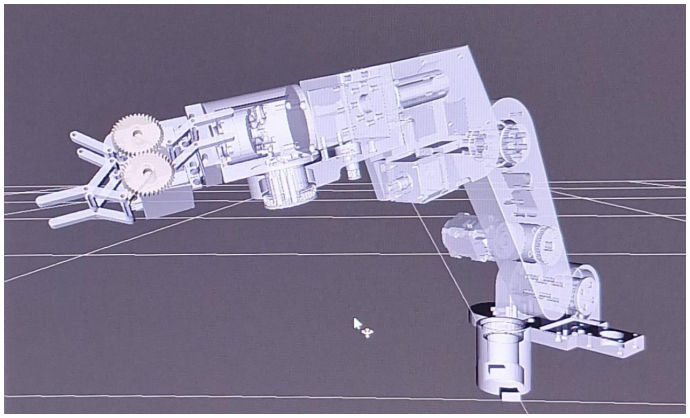
Entwicklung eines 6dof Roboterarms



ASIT-Projekt verfasst von

Matteo Gassler
Emanuel Käfer
Alexander Dick

Betreuer: Prof. Dr. Gerhard Gaube

HTBLA Saalfelden

Abteilung für Elektrotechnik

Abgabevermerk:

Saalfelden, 19 Juni 2026 übernommen von:

Inhaltsverzeichnis

1 Projektüberblick **1**

1.1 Zielsetzung 1

1.2 Umsetzung 1

1.3 Umsetzungsstand der Software 1

2 Low Level Architektur und Steuerung **2**

2.1 Implementierung der 6 Achsen und Skalierbarkeit 2

2.2 Initialisierung und SPI-Kommunikation der Motortreiber 4

2.3 Finite State Machine (Endlicher Zustandsautomat) 7

2.4 Kinematische Referenzierungsfahrt (Homing) 8

2.5 Ausblick 10

3 High Level Software **11**

3.1 Ausblick 11

4 Hardware **12**

5 Elektrisches **13**

Quellenverzeichnis **15**

1 Projektüberblick

1.1 Zielsetzung

Da die Entwicklung unseres 6dof Roboterarms im Rahmen des Faches ASIT erfolgt und der Lernerfolg im Vordergrund steht konnten wir uns die Aufgabenstellung selbst stellen und hatten dabei keine großen Einschränkungen. Wir haben uns dazu entschieden das der Roboterarm als Mindestanforderung eine Pick and Place Operation, also ein Objekt aufheben und wo anders platzieren können soll. Als optionale weitere Herausforderung soll man über ein Graphical user Interface Koordinaten eingeben können zu denen sich der Arm dann bewegt. Nach einem Schuljahr Arbeit an dem Projekt haben wir uns entschieden es als Diplom und Abschlussarbeit fortzusetztn.

1.2 Umsetzung

1.3 Umsetzungsstand der Software

Der aktuelle Umsetzungsstand der Software umfasst ein vollständiges Initialisieren der Motortreiber und ein sequentielles Homing. Dieses sequentielle Homing wird im verlauf des Projekts aber noch durch ein Absolut encoderhoming ersetzt.

2 Low Level Architektur und Steuerung

Dieses Kapitel beschreibt die Implementierung der zentralen Low-Level-Robotersteuerung, die auf einem Mikrocontroller läuft. Sie stellt das funktionale Bindeglied zwischen der mechanischen Konstruktion (siehe 4) und der elektrischen Leistungselektronik (siehe 5) dar. Die Firmware wurde auf Hardwareabstraktionsebene (STM32CubeHAL) basierendem C für einen STM32F446-Mikrocontroller in der Entwicklungsumgebung STM32CubeIDE entwickelt [4]. Ziel der Softwarearchitektur ist es, harte Echtzeitanforderungen für die synchrone Ansteuerung von sechs Schrittmotoren zu erfüllen und gleichzeitig eine hochgradig skalierbare Basis für zukünftige Pfadplanungen zu schaffen.

2.1 Implementierung der 6 Achsen und Skalierbarkeit

Bei der Ansteuerung eines 6-Achs-Roboterarms führt die Zuweisung von Mikrocontroller-Pins als Einzelvariablen und die statische Zuweisung von Peripherie-Registern schnell zu eingeschränktem und fehleranfälligem Programmcode. Um dies zu vermeiden, wurde eine Pointer-Array-basierte Datenstruktur implementiert. Anstatt jede Achse separat zu adressieren, werden Pointer auf die entsprechenden Hardware-Timer (`TIM_HandleTypeDef`), PWM-Kanäle sowie die GPIO-Ports für Richtungs- (`DIR`) und Chip-Select-Signale (`CS`) in globalen Arrays gekapselt.
Durch diese Entkopplung von physikalischer Hardware und Steuerungslogik kann jede Achse über einen simplen Index angesprochen werden. Dies ermöglicht es, Kernfunktionen wie das Starten und Stoppen der Motoren oder das Homing über alle Treiber mittels einfacher Schleifenkonstrukte umzusetzen, was die Skalierbarkeit und Wartbarkeit des Systems maßgeblich erhöht.

2.1.1 Richtungssignale (DIR-Pins)

Zur Bestimmung der Drehrichtung benötigt jeder Schrittmotortreiber einen DIR-Pin (Direction). Ein High-Pegel (1) bewirkt eine Drehung im Uhrzeigersinn, ein Low-Pegel (0) eine Drehung gegen den Uhrzeigersinn. Um dies für alle sechs Motoren effizient

und skalierbar umzusetzen, wurden Arrays eingesetzt.

```
1 GPIO_TypeDef* DIR_PORTS[6] = {TMC1_DIR_GPIO_Port, TMC2_DIR_GPIO_Port,
    TMC3_DIR_GPIO_Port, TMC4_DIR_GPIO_Port, TMC5_DIR_GPIO_Port,
    TMC6_DIR_GPIO_Port};
2 uint16_t       DIR_PINS[6]   = {TMC1_DIR_Pin, TMC2_DIR_Pin,  TMC3_DIR_Pin
    , TMC4_DIR_Pin, TMC5_DIR_Pin, TMC6_DIR_Pin};
```

Programmcode 2.1: Arrays für DIR-Pins/Ports

2.1.2 Geschwindigkeitsregelung (STEP-Pins)

Die Geschwindigkeitsregelung über die STEP-Pins wird mittels Hardware-Pulsweitenmodulation (PWM) durch Timer implementiert. Die Motortreiber benötigen zur Vorgabe der Geschwindigkeit ein Frequenzsignal am STEP-Eingang. Dies ist ebenfalls durch Arrays realisiert.

```
1 TIM_HandleTypeDef* AXIS_TIMERS[6] = {&htim2, &htim1, &htim3, &htim4, &
    htim8, &htim10};
2 uint32_t AXIS_CHANNELS[6] = {TIM_CHANNEL_1, TIM_CHANNEL_1,
    TIM_CHANNEL_1, TIM_CHANNEL_1, TIM_CHANNEL_1, TIM_CHANNEL_1};
```

Programmcode 2.2: Arrays für STEP-Timer/Channels

2.1.3 Vergleich: Mit und ohne Array-Abstraktion

Ohne Array-Abstraktion würde die Motoransteuerung mehrere if-else-Blöcke erfordern. Als Beispiel wird die Funktion zum Stoppen einer Motor-PWM von drei Achsen gezeigt.

Hardcodierte Hardware-Zuordnung

```
1 void Step_Stop_HardCoded(uint8_t ax) {
2  if (ax == 0) {
3   HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1);
4   HAL_TIM_Base_Stop_IT(&htim2);
5  }
6  else if (ax == 1) {
7   HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
```

```
8     HAL_TIM_Base_Stop_IT(&htim1);
9    }
10   else if (ax == 2) {
11    HAL_TIM_PWM_Stop(&htim3, TIM_CHANNEL_1);
12    HAL_TIM_Base_Stop_IT(&htim3);
13   }
14
15  }
```

Programmcode 2.3: Motor-Stopp ohne Array-Abstraktion (Anti-Pattern)

Das führt zu mehreren Problemen:

- **Code-Duplikation**: Die gleiche Logik wird sechsmal wiederholt.
- **Wartbarkeit**: Änderungen müssen sechsmal vorgenommen werden.
- **Fehleranfälligkeit**: Es ist leicht, einen Achsen-Fall zu vergessen oder falsch zuzuordnen.
- **Skalierbarkeit**: Das Hinzufügen oder Entfernen einer Achse erfordert massive Codeänderungen.

2.2 Initialisierung und SPI-Kommunikation der Motortreiber

2.2.1 SPI-Protokoll

Das Serial Peripheral Interface (SPI) ist ein serielles Hochgeschwindigkeits-Bussystem, das vorwiegend zur Kommunikation von Mikrocontrollern mit Peripheriegeräten auf Platinen dient. SPI ist vollduplexfähig, das heißt, dass bei jedem Takt ein Bit vom Master an den Slave und umgekehrt gesendet wird. Es besteht aus drei Leitungen: MOSI (Master Out $\longrightarrow$ Slave In), MISO (Master In $\longleftarrow$ Slave Out) und SCK (Serial Clock). Hinzu kommt eine CS-Leitung (Chip Select), die vom Master separat zu jedem Slave geführt werden muss. Nur wenn diese CS-Leitung auf LOW gezogen wird, sendet oder empfängt der Slave Daten. Die Kommunikation erfordert in der Regel zwei Übertragungszyklen. Im ersten Zyklus sendet der Master ein Kommando an den Slave. Da der Slave zu diesem Zeitpunkt die Antwort noch nicht vorbereitet hat, sendet er irrelevante Dummy-Daten zurück. In der zweiten Übertragung sendet der Master

einen leeren Datensatz, um das Taktsignal aufrechtzuerhalten, woraufhin der Slave die bereitgestellte Antwort übermittelt. Die Übertragungen erfolgen jeweils synchron [1].

SPI-Modi

Es existieren vier SPI-Modi (0-3). Diese definieren sich wie folgt:

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Abbildung 2.1: Diagramm der SPI-Modi

CPOL (Clock Polarity):

- 0: Takt ist im Ruhezustand LOW, ein Wechsel auf HIGH zählt als steigende Taktflanke.
- 1: Takt ist invertiert (im Ruhezustand HIGH), ein Wechsel auf LOW zählt als steigende Taktflanke.

CPHA (Clock Phase):

- 0: Daten werden bei der ersten Taktflanke (abhängig von CPOL) eingelesen und bei der darauffolgenden ausgegeben.
- 1: Daten werden bei der zweiten Taktflanke eingelesen und bei der ersten ausgegeben.

2.2.2 Initialisierung der Motortreiber

Die verwendeten Schrittmotortreiber vom Typ TMC5160 in Kombination mit NEMA-17-Schrittmotoren erfordern eine initiale Konfiguration ihrer internen Register, um Parameter wie Haltestrom, Betriebsstrom (Run-Current) und den Chopper-Modus festzulegen. Diese Kommunikation erfolgt über einen geteilten SPI-Bus. Um die Treiber individuell adressieren zu können, wird vor jedem Datentransfer der spezifische Chip-Select-Pin (`CS`) der Zielachse auf ein niedriges Potential (LOW) gezogen. Die TMC5160 erfordern den Betrieb im SPI-Modus 3 (siehe 2.1) [2].

Um Fehler direkt bei der Initialisierung der Treiber zu erkennen, wurde eine Validierung implementiert, die zwingend erfolgreich durchlaufen werden muss. Nach dem Schreiben der Konfigurationsdaten liest der Mikrocontroller das globale Statusregister (`GSTAT`) sowie aktuelle Treiberzustände zurück. Nur wenn das System valide Antworten (`GSTAT` = 0) detektiert, wird die Steuerung freigegeben. Störsignale oder fehlende Hardware-Verbindungen werden so unmittelbar beim Systemstart erkannt und führen zu einem sicheren Systemabbruch.

Die gewählte Konfiguration der Motortreiber stellt sich wie folgt dar:

Tabelle 2.1: Konfiguration der TMC5160T-Register via SPI

Register (Wert)	**Funktion**
`GSTAT` (0x01) = `0x00000007`	Setzt das globale Statusregister zurück. Löscht den Fehlerspeicher für Reset, Unterspannung der Charge-Pump und Driver-Shutdown vor dem Testlauf.
`GCONF` (0x00) = `0x00000004`	Aktiviert das Bit `pdn_disable`. Schaltet den Treiber in den internen Sensing-Modus um und erzwingt die Kommunikation rein über die SPI-Schnittstelle.
`IHOLD_IRUN` (0x10) = `0x00061008`	Konfiguriert den Motorstrom: Setzt den Fahrstrom (`IRUN`) auf ca. 50 %, den Haltestrom im Stillstand (`IHOLD`) auf ca. 25 % sowie eine Verzögerung (`IHOLDDELAY`) für die Stromabsenkung.
`CHOPCONF` (0x6C) = `0x14010053`	Konfiguration des Chopper-Verfahrens. Aktiviert den *SpreadCycle*-Modus, stellt die Schrittauflösung auf 16 Mikroschritte (mit automatischer Interpolation auf 256 Schritte) und definiert das Signal-Timing (*TOFF*, *TBL*).
`PWMCONF` (0x70) = `0xC10D0024`	Hinterlegt die Parameter für den *StealthChop2*-Modus (Spannungsamplitude, Frequenz, Gradienten) für einen potenziellen, geschwindigkeitsabhängigen Moduswechsel.

[2]

2.3 Finite State Machine (Endlicher Zustandsautomat)

Um die auf dem Mikrocontroller laufende Software deterministisch zu gestalten und Deadlocks zu verhindern, kommt eine Finite State Machine (FSM) zum Einsatz. Eine FSM ist ein mathematisches Modell für Systeme, die sich zu jedem Zeitpunkt in genau einem von mehreren Zuständen befinden. Eine FSM besteht strukturell aus den folgenden Komponenten [3]:

- Zustände (States): Die verschiedenen operativen Phasen des Systems.
- Startzustand (Initial State): Der Zustand, in dem die FSM initialisiert wird.
- Übergänge (Transitions): Die definierten Bedingungen für den Wechsel zwischen Zuständen.
- Ereignisse (Inputs): Die Trigger, die einen Zustandswechsel bewirken.
- Aktionen (Outputs): Die ausführenden Operationen der FSM.

2.3.1 Zustände

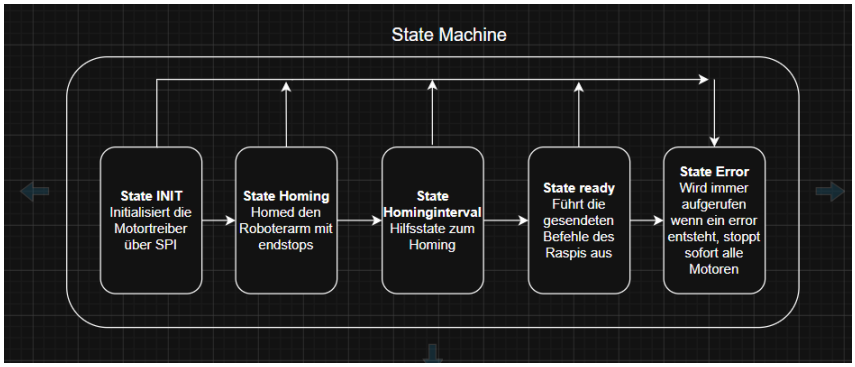


Abbildung 2.2: Diagramm der Finite State Machine

Für den entwickelten Roboterarm wurden die Zustände `STATE_INIT`, `STATE_HOMING`, `STATE_HOMING_OFFSET`, `STATE_READY` und `STATE_ERROR` definiert. Als Startzustand dient `STATE_INIT`, da vor dem regulären Betrieb eine Treiberinitialisierung zwingend erforderlich ist. Die Spezifikationen der Zustände gestalten sich wie folgt:

Tabelle 2.2: Zustände der Finite State Machine

Zustand	Funktion
STATE_INIT	Initialisiert die Motortreiber über den SPI-Bus und konfiguriert die Registerparameter.
STATE_HOMING	Führt den ersten Teil der kinematischen Referenzfahrt durch, bei der die Achse bis zur Auslösung des Endschalters verfahren wird.
STATE_HOMING_OFFSET	Führt den zweiten Teil der kinematischen Referenzfahrt durch. Die Achse verfährt vom Endschalter um einen definierten Offset auf die finale Home-Position.
STATE_READY	Der reguläre Betriebszustand. Der Mikrocontroller verarbeitet eingehende Befehle und setzt diese in Trajektorien um.
STATE_ERROR	Fehlerzustand. Alle Motoren werden umgehend gestoppt und ein Fehlercode wird ausgegeben. Ein manueller Reset ist erforderlich.

2.4 Kinematische Referenzierungsfahrt (Homing)

Da Schrittmotoren über keine absolute Positionscodierung verfügen und aus Kostengründen auf absolute Encoder verzichtet wurde, ist der Systemzustand nach jedem Kaltstart undefiniert. Um den vordefinierten kinematischen Nullpunkt des Roboters zu ermitteln, muss zwingend eine Kalibrierungsfahrt (Homing) durchgeführt werden.

2.4.1 Interrupt-Verarbeitung und Hardware-Entprellung

Für die Erfassung der Endschaltersignale kommen externe Interrupts (EXTIs) zum Einsatz. Die Verwendung von Interrupts reduziert die Prozessorauslastung signifikant, da aktives Polling (permanentes Abfragen der Pins) entfällt. Sobald eine Signalflanke detektiert wird, löst der Hardware-Controller einen Interrupt-Request aus, der den regulären Programmablauf unterbricht und eine spezifische Interrupt-Service-Routine (ISR) aufruft. Diese Routine setzt ein Flag, das dem Hauptprogramm signalisiert, dass der Endschalter erreicht wurde, der Positions-Offset berechnet und die Drehrichtung umgekehrt werden muss [4].
Um mehrfache Interrupt-Auslösungen durch mechanisches Schalterprellen (Bouncing) zu verhindern, ist in der Interrupt-Routine eine softwareseitige Entprellung (Debouncing) integriert. Diese blockiert nach der ersten Flankenerkennung alle weiteren Signale desselben Pins für ein Zeitfenster von 30 ms.
Die Implementierung in C ist wie folgt realisiert:

```
static uint32_t last = 0;
uint32_t now = HAL_GetTick();
if (now - last < 30) return;
last = now;
```

Programmcode 2.4: Software-Entprellung (Debouncing) innerhalb der ISR

2.4.2 Berechnung kinematischer Offsets

Der mechanische Nullpunkt an der Position der Endschalter entspricht in der Regel nicht dem kinematischen Nullpunkt des Robotermodells. Diese Diskrepanz erfordert die Berechnung eines Ausgleichsoffsets. Die zu kompensierende Winkeldifferenz muss dazu in die entsprechende Schrittanzahl der Motoren umgerechnet werden. Die allgemeine Gleichung lautet:

$$\text{Steps} = \frac{\Delta\text{rad}}{2 \cdot \pi} \cdot \text{Vollschritte} \cdot \text{Mikroschritte} \cdot \text{Getriebeübersetzung}$$

Mit den konkret eingesetzten Hardwareparametern ergibt sich folgende Berechnung:

$$\text{Steps} = \frac{\Delta\text{rad}}{2 \cdot \pi} \cdot 200 \cdot 16 \cdot 20$$

- Δrad: Die berechnete Winkeldifferenz zwischen Endschalter und kinematischer Home-Position (in Radiant).
- 200: Die Anzahl der physikalischen Vollschritte des Motors pro Umdrehung ($1,8^\circ$ Schrittwinkel).
- 16: Die konfigurierte Mikroschritt-Auflösung pro Vollschritt.
- 20: Das mechanische Untersetzungsverhältnis des Getriebes (1:20) für alle Achsen.

2.4.3 Sequenzieller Homing-Ablauf

Die Referenzierung des 6-Achs-Systems erfolgt aus Sicherheitsgründen sequenziell. Dadurch werden mechanische Kollisionen oder kritische Fehler – wie das gleichzeitige Auffahren mehrerer Achsen auf die Endanschläge – konstruktiv ausgeschlossen. Ein weiterer Vorteil des sequenziellen Homings ist die Möglichkeit der isolierten visuellen Überprüfung der Korrektheit jeder einzelnen Achsbewegung. Der Ablauf wird durch

eine globale Index-Variable (`current_homingaxis`) gesteuert und verläuft hierarchisch von der Basis (Achse 0) bis zum Endeffektor (Achse 5). Der Vorgang pro Achse gliedert sich in folgende Phasen:
Zunächst verfährt die Achse im Zustand `STATE_HOMING` mit konstanter Geschwindigkeit in Richtung des Endschalters. Nach Auslösen des Interrupts berechnet der Algorithmus den erforderlichen Offset, invertiert das Richtungssignal (`DIR`) und leitet den Zustandswechsel zu `STATE_HOMING_OFFSET` ein. Die Achse gilt erst dann als referenziert, wenn die berechnete Schrittzahl für den Offset über den Timer-Interrupt exakt abgefahren wurde. Anschließend wird der Achsindex inkrementiert und die Prozedur für das nächste Gelenk gestartet, bis das Gesamtsystem kalibriert ist und in den regulären Betriebszustand `STATE_READY` übergeht.

2.5 Ausblick

Die nächsten Evaluierungsschritte für die Low-Level-Logik umfassen die Umstellung des Homing-Prozesses von Endschaltern auf eine kontinuierliche Winkelerfassung mittels Encodern. Des Weiteren muss eine robuste Kommunikationsschnittstelle zwischen dem Raspberry Pi, der die High-Level-Trajektorien berechnet, und dem Mikrocontroller etabliert werden. Der finale Meilenstein zur Fertigstellung der Low-Level-Steuerung ist die Integration eines trapezförmigen Geschwindigkeitsprofils, um ein ruckfreies Beschleunigen und Abbremsen der Gelenke sicherzustellen.

3 High Level Software

3.1 Ausblick

Die High-Level-Software auf dem Raspberry Pi ermöglicht die Eingabe kartesischer Zielkoordinaten für den Endeffektor über ein Graphical User Interface (GUI). Diese kartesischen Koordinaten (x, y, z) werden mithilfe der Python-Bibliothek IKpy über inverse Kinematik in sechs Gelenkwinkel in Radiant transformiert. Zudem lässt sich über die GUI die maximale Geschwindigkeit definieren, auf welche die Motoren beschleunigen dürfen. Da der Fokus der bisherigen Entwicklung primär auf der Low-Level-Software lag, befindet sich die High-Level-Implementierung noch in einer frühen Projektphase.

4 Hardware

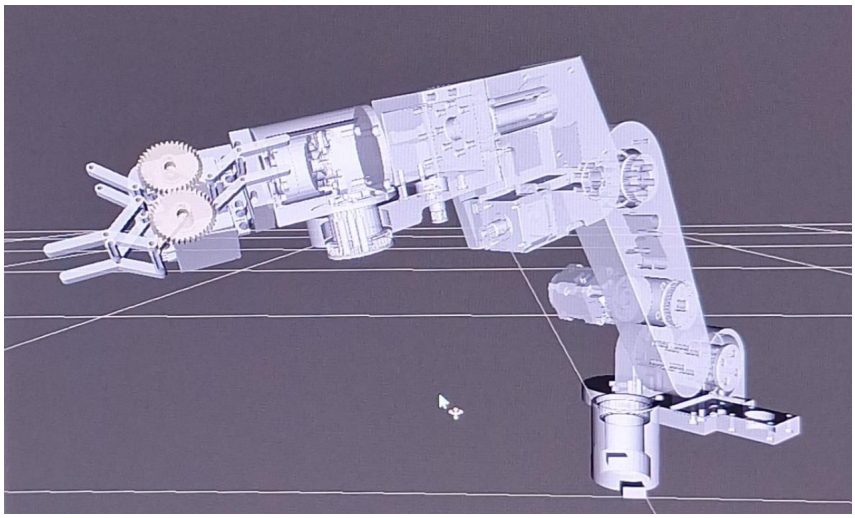


Abbildung 4.1: Blockdiagramm des 6-DOF-Roboterarms

5 Elektrisches

Die Entwicklung moderner Roboterarme basiert stark auf den Prinzipien der kollaborativen Robotik,[5] bei denen Sicherheit und Flexibilität im Vordergrund stehen [**ws2812**].

Abbildungsverzeichnis

2.1 Diagramm der SPI-Modi 5
2.2 Diagramm der Finite State Machine 7

4.1 Blockdiagramm des 6-DOF-Roboterarms 12

Quellenverzeichnis

[1] *Serial Peripheral Interface – Mikrocontroller.net*, Mikrocontroller.net, 2025. Adresse: https://www.mikrocontroller.net/articles/Serial_Peripheral_Interface

[2] *POWER DRIVER FOR STEPPER MOTORS INTEGRATED CIRCUITS TMC5160 / TMC5160A DATASHEET FEATURES AND BENEFITS 2-phase stepper motors from 1 to several 10A coil current Motion Controller with SixPoint™ ramp*. Adresse: https://www.analog.com/media/en/technical-documentation/data-sheets/TMC5160A_datasheet_rev1.17.pdf

[3] *Statemachine – Mikrocontroller.net*, Mikrocontroller.net, 2026. besucht am 27. Mai 2026. Adresse: https://www.mikrocontroller.net/articles/Statemachine

[4] *STM32F446RE Datasheet*, 2021. Adresse: https://www.st.com/resource/en/datasheet/stm32f446mc.pdf

[5] *WS2812B Intelligent control LED integrated light source Features and Benefits*. Adresse: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf